# JUnit

# JUnit4 Annotations

| Annotation | Description |
|---|---|
| `@Test public void method()` | The `Test` annotation indicates that the public void method to which it is attached can be run as a test case. |
| `@Before public void method()` | The `Before` annotation indicates that this method must be executed before each test in the class, so as to execute some preconditions necessary for the test. |
| `@BeforeClass public static void method()` | The `BeforeClass` annotation indicates that the static method to which is attached must be executed once and before all tests in the class. That happens when the test methods share computationally expensive setup (e.g. connect to database). |
| `@After public void method()` | The `After` annotation indicates that this method gets executed after execution of each test (e.g. reset some variables after execution of every test, delete temporary variables etc) |
| `@AfterClass public static void method()` | The `AfterClass` annotation can be used when a method needs to be executed after executing all the tests in a JUnit Test Case class so as to clean-up the expensive set-up (e.g disconnect from a database). Attention: The method attached with this annotation (similar to `BeforeClass`) must be defined as static. |
| `@Ignore public static void method()` | The `Ignore` annotation can be used when you want temporarily disable the execution of a specific test. Every method that is annotated with `@Ignore` won't be executed. |

| JUNIT 4 ANNOTATION | JUNIT 5 ANNOTATION |
| --- | --- |
| @Before | @BeforeEach |
| @After | @AfterEach |
| @BeforeClass | @BeforeAll |
| @AfterClass | @AfterAll |
| @Test | @Test |

# JUnit assertions

| Assertion | Description |
|---|---|
| `void assertEquals([String message], expected value, actual value)` | Asserts that two values are equal. Values might be type of int, short, long, byte, char or java.lang.Object. The first argument is an optional String message. |
| `void assertTrue([String message], boolean condition)` | Asserts that a condition is true. |
| `void assertFalse([String message],boolean condition)` | Asserts that a condition is false. |
| `void assertNotNull([String message], java.lang.Object object)` | Asserts that an object is not null. |
| `void assertNull([String message], java.lang.Object object)` | Asserts that an object is null. |
| `void assertSame([String message], java.lang.Object expected, java.lang.Object actual)` | Asserts that the two objects refer to the same object. |
| `void assertNotSame([String message], java.lang.Object unexpected, java.lang.Object actual)` | Asserts that the two objects do not refer to the same object. |
| `void assertArrayEquals([String message], expectedArray, resultArray)` | Asserts that the array expected and the resulted array are equal. The type of Array might be int, long, short, char, byte or java.lang.Object. |

```
        @Test
        public void test() {
                String obj1 = "junit";
                String obj2 = "junit";
                String obj3 = "test";
                String obj4 = "test";
                String obj5 = null;
                int var1 = 1;
                int var2 = 2;
                int[] arithmetic1 = { 1, 2, 3 };
                int[] arithmetic2 = { 1, 2, 3 };

                assertEquals(obj1, obj2);

                assertSame(obj3, obj4);

                assertNotSame(obj2, obj4);

                assertNotNull(obj1);

                assertNull(obj5);

                assertTrue(var1  var2);

                assertArrayEquals(arithmetic1, arithmetic2);
        }

}
```

| JUNIT 4 ANNOTATION | JUNIT 5 ANNOTATION | Description in brief |
|---|---|---|
| @FixMethodOrder | @TestMethodOrder & @Order | 1. These annotations allow the user to choose the order of execution of the methods within a test class |
| @Rule & @ClassRule | @ExtendWith | 1. @Rule – The annotation is extended from the class TestRule that helps apply certain rules on the test cases.<br>2. For Example: creating a temporary folder prior to test case execution and deleting the folder post-execution can be set through a Rule.<br>3. @Rule is available only in JUnit 4 which can be used in JUnit 5 Vintage, however, @ExtendWith provides a closer feature for JUnit 5<br>4. Similarly, a global timeout can be set using @Rule. |
| NA | @TestFactory | 1. This annotation supported by JUnit 5 only and helps creation of dynamic or runtime tests.<br>2. It returns a stream of data as collection and cannot use lifecycle callback annotations |
| NA | @Nested | 1.This annotation is supported by JUnit Jupiter only<br>2.It helps us to create nested test cases.<br>3.For instance, Class 1 with testcase 1 might have a @Nested Class 2 with testcase 2. This makes testcase 2 a nested testcase to testcase 1. Hence, testcase 1 executes, then testcase 2 executes.<br>4.If the @Nested annotation is not used the nested class will not execute. |

| | | |
|---|---|---|
| @Category | @Tag | 1.This annotation helps for tagging and filtering the tests<br>2.You may include tests for execution or exclude them by filtering based on the categories they fall in. |
| @RunWith(Parameterized.class)<br>@Parameterized.Parameters | @ParameterizedTest and @ValueSource | 1. This annotation is used to run a method with test data variations multiple times.<br>2.JUnit 4 supports @RunWith and @Parameters while JUnit 5 Jupiter supports @ParameterizedTest with @ValueSource |
| | @RepeatedTest | 1.JUnit 5 supports repeated execution of the test method for a certain number of times using @RepeatedTest annotation |
| | @DisplayName | 1. A user-defined name can be given to a test method or class for display purposes. |
| | @TestInstance (LifeCycle.PER_CLASS) and @TestInstance (LifeCycle.PER_METHOD) | 1. JUnit 5 supports the configuration of the lifecycle of tests.<br>2. Both JUnit 4 and 5 follow the default per method lifecycle call-back while per-class configuration can also be done. |